Delo z mapami in datotekami

Najpogostejše težave, s katerimi se boste srečevali pri programiranju in niso povezane s Pythonom, so:

- program ne najde datoteke,
- testi ne najdejo vašega programa,
- testi kljub spremembam javljenih napak še vedno vračajo iste napake in
- spremembe v vaši datoteki so izginile.

Naštete težave so povezane z lokacijami map in datotek, s katerimi delate.

Datotečni sistem

Datotečni sistem (angl. *file system*) hrani in ureja datoteke ter zagotavlja njihovo dostopnost. Datoteke shranjujemo v mapah (pogost izraz tudi: direktorij, angl. *directory*), ki so lahko del drugih map. S tem nastane hierarhična struktura, ki jo na papirju izrisujemo s shemo:

```
C:\
|-- Program Files
|-- Windows
`-- Users
    `-- student
        |-- Desktop
        |-- Documents
        `-- Downloads
```

Prvi vnos v zgornji shemi je **korenska mapa** (angl. *root folder*), `C:\`, ki nima nadrejene mape. Znotraj nje se nahajajo podmape (angl. *subdirectory*), ki lahko vsebujejo druge mape. Mapa `Users` je nadrejena (angl. *parent folder*) mapi `student`. Z drugimi besedami: mapa `student` je podmapa (subdirektorij) mape `Users`. Ker mapa `student` vsebuje druge podmape, je ta nadrejena subdirektorijem `Desktop`, `Documents` in `Downloads`.

Znotraj korenske mape se shranjujejo vse datoteke in mape, ki jih imamo na računalniku. V operacijskem sistemu (OS) Windows ima vsak disk svojo korensko mapo, ki se označi s črko diska, npr. `C:\`, ko v sistem vključimo nov disk, kot sta zunanji disk ali USB ključek, dobi ta disk svojo korensko mapo, npr. `D:\`.

OS macOS in Linux imata le eno samo korensko mapo, znotraj katere se pojavijo zunanji diski in USB ključki. Korenska mapa je v teh dveh sistemih označena s poševnico `/`.

Domača mapa (angl. *home folder*) je mapa, v kateri se hranijo datoteke, povezane z določenim uporabniškim računom. Lokacije te mape glede na OS so:

- Windows: `C:\Users\student`
- macOS: `/Users/student`[1]
- Linux: `/home/student`[1]

Pot do datoteke

Pot (angl. *path*) opisuje pot do želene datoteke ali mape in je sestavljena iz zaporednih imen map, ločenih z ločilom - Windows uporablja \, Linux in macOS pa /. Ker ima znak \ v nizih v Pythonu poseben pomen, poti v sami kodi raje pišemo z znakom /, ki bo deloval ne glede na OS.

```
C:\
|-- Program Files
|-- Windows
`-- Users
    `-- student
        |-- Desktop
        |-- Documents
        |   `-- naloge.py
        `-- Downloads
```

Pot do datoteke `naloge.py` bo tako v OS:

- Windows: `C:\Users\student\Documents\naloge.py`
- macOS: `/Users/student/Documents/naloge.py`
- Linux: `/home/student/Documents/naloge.py`

Kadar opišemo celotno pot od korenske mape do želene datoteke, smo zapisali **absolutno pot**.

Delovna mapa

Delovna mapa/imenik (angl. *working directory*) je mapa, v kateri se program trenutno „nahaja“. Če delamo z datoteko `naloge.py` iz zgornjega primera, bo naša delovna mapa `Documents`.

Relativna pot do datoteke vsebuje le podmape delovne mape - če je naša delovna mapa `Documents`, mi pa v skladu s spodnjo shemo želimo odpreti datoteko `vaja.py`, ki se nahaja v mapi `Vaje`, bo relativna pot enaka `\Vaje\vaja.py`.

Če pred delovno mapo zapišemo `.`, to pomeni, da ostajamo v delovni mapi. Če zapišemo `..`, Python usmerimo mapo, nadrejeno trenutni delovni mapi.

Če želimo datoteko odpreti z relativno potjo, moramo najprej poznati našo delovno mapo, sicer Python ne bo nujno našel želene datoteke.

[1] Pogosto skrajšano z znakom `~`: `~/Desktop`.

Če naš program v IDLE zaženemo z ukazom *Run Module* (F5), bo IDLE za delovno mapo določil mapo z našim programom. Če ukaz `os.getcwd()` vpišemo v lupino IDLE, ki je še nismo uporabili za zagon programa, bo delovna mapa nekje drugje.

Na vajah pri zagonu testov ali uporabi zunanjih datotek (9. vaje) pogosto opazimo napako `FileNotFoundError`, ki skoraj vedno pomeni, da je datoteka bodisi v drugi mapi in ne v trenutnem delovnem imeniku, ali pa je poimenovana drugače kot mislimo. V takšnem primeru preverite, kje je vaš trenutni delovni imenik in katere datoteke se nahajajo znotraj njega.

Preverjanje lokacije datoteke ali mape

V **Raziskovalcu** (OS Windows): kliknite v naslovno vrstico, kjer se prikaže celotna pot do trenutne mape. Pot do datoteke kopirate z desnim klikom nanjo in nato v spustnem seznamu izberete *Kopiraj kot pot*/*Copy as path* (Ctrl+Shift+C).

V **Finderju** (macOS) vklopite *View* → *Show Path Bar*, ki na dnu okna prikaže pot do trenutne mape. Pot do datoteke z desnim klikom nanjo in v spustnem seznamu izberete *Copy ... as Pathname*.

Ko datoteko odprete **v IDLE**, je celotna pot do nje zapisana v naslovni vrstici okna. Pred zagonom testov vedno preverite, ali je datoteka v mapi s testi – za orientacijo vedno poiščite datoteko `test.py`.

Lokacijo programa, delovne mape in vsebine delovne mape lahko zakodiramo znotraj naše datoteke in jih s funkcijo `print` izpišemo na zaslon:

```
1 import os
2 print(__file__)        # pot do datoteke s programom
3 print(os.getcwd())     # delovna mapa
4 print(os.listdir())    # vsebina delovne mape
```

Imena datotek in map

Pri poimenovanju datotek in map:

- ne uporabljamo presledkov, temveč podčrtaje,
- ne uporabljamo šumnikov, vejic, pik in drugih posebnih znakov (č, š, ž, `&`, `#`, ...),
- poskrbimo, da je ime datoteke z rešitvijo točno takšno, kot ga zahtevajo navodila in v mapi, v kateri je shranjena datoteka `test.py` in pripadajoče mape, sicer testi ne bodo delovali,
- pazimo na uporabo velikih in malih črk – v `Naloga.py` in `naloga.py` sta v OS Windows in macOS isti datoteki, v Linux pa ne[2].

[2]OS Windows po privzetih nastavitvah pri datotekah ne razlikuje velikih in malih črk, medtem ko jih Linux razlikuje. To pomeni, da koda, ki deluje v Windows morda ne bo delovala v Linux (in obratno), če nismo pozorni na velike in male črke.

Končnice datotek

Končnica datoteke (angl. *file extension*) je del imena za piko, preko katere OS ugotovi, s katerim programom se datoteka odpira. Tako Windows kot macOS po privzetih nastavitvah končnice skrivata, zato se pogosto zgodi, da je datoteka, ki jo vidimo kot `naloga.py` v resnici `naloga.py.py`.

Ogled končnic datotek omogočite:

- Windows: v Raziskovalcu izberite *Pogled* → *Pokaži* → *Pripone imen datotek* (*View* → *Show* → *File name extensions*)
- macOS: v Finderju izberite *Finder* → *Settings* → *Advanced* in odkljukajte *Show all filename extensions*.

Datoteke s programi shranjujte neposredno iz IDLE (*File* → *Save As*), ki avtomatsko doda končnico `.py` in vam je ni treba vpisovati v polje za poimenovanje.

Stisnjene mape in ekstrakcija

Stisnjena mapa ima končnico `.zip`, v njej pa so zapakirane druge datoteke in/ali mape. S stiskanjem nekoliko zmanjšamo velikost mape in omogočimo hitrejše deljenje preko spleta. Testne skripte in gradiva z GitHuba boste vedno prejeli v obliki stinjenih map, ki pa jih morate pred začetkom dela v mapi ekstrahirati (razširiti, angl. *extract*), sicer tvegate nedelovanje testov in izgubo podatkov.

Ker OS Windows omogoča vpogled v stisnjene mape[3], pred začetkom dela vedno preverite, da ste mapo res ekstrahirali. V izogib shranjevanju v napačno mapo priporočam, da stisnjeno mapo po prenosu v `Prenosi/Downloads` ekstrahirate v `Namizje/Desktop` (ciljna mapa), v `Prenosi/Downloads` pa jo izbrišete.

Če opazite, da datoteke ne delajo, preverite pot do datoteke, ki v primeru stisnjene mape vsebuje `AppData\Local\Temp`. Če to opazite, datoteko, s katero delate, zaprite, mapo ekstrahirajte in delo nadaljujte v ekstrahirani mapi.

Ekstrahiranje mape v OS Windows izvedemo tako, da z desnim klikom kliknemo na mapo in v meniju izberemo *Ekstrahiraj vse*/*Extract all*. Odprlo se bo novo okence, kjer lahko določite ciljno mapo ekstrahiranih datotek.

V macOS ekstrakcijo izvedete z dvoklikom na mapo. Ko je ta enkrat ekstrahirana, jo sami premaknete v želeno ciljno mapo.

[3] macOS vpogleda v stisnjene mape ne omogoča.

Splošne bližnjice na tipkovnici

Zapisane bližnjice delujejo v večini programov in pogosto pohitrijo delo.

Windows	macOS	Pomen
Ctrl+C / Ctrl+V	Cmd+C / Cmd+V	kopiraj/prilepi
Ctrl+X	Cmd+X	izreži
Ctrl+Z	Cmd+Z	razveljavi
Ctrl+S	Cmd+S	shrani
Ctrl+A	Cmd+A	izberi vse
Ctrl+F	Cmd+F	iskanje besedila
Ctrl+H	Cmd+F	išči in zamenjaj (angl. *find and replace*)
Alt+Tab	Cmd+Tab	preklop med aktivnimi okni
Win+E	Cmd+Space	Raziskovalec / iskanje Spotlight

Bližnjice do znakov v OS Windows[4], ki jih boste potrebovali pri programiranju:

Znak	Windows
[	`AltGr + F`
]	`AltGr + G`
{	`AltGr + B`
}	`AltGr + N`
\	`AltGr + Q`
|	`AltGr + W`
`	`AltGr + 7`

Osnove terminala

Terminal [5] omogoča delo z datotečnim sistemom preko ukazov. Začetna delovna mapa je domača mapa, z ukazi pa se lahko pomikamo po datotečnem sistemu in spremenimo delovno mapo. Uporabni ukazi pri delu v terminalu so:

Windows	macOS/Linux	Pomen
`cd`	`pwd`	izpis trenutne mape (angl. *current directory* oz. *print working directory*
`dir`	`ls`	izpis vsebine mape
`cd ime_mape`	`cd ime_mape`	premik v podmapo
`cd ..`	`cd ..`	premik v nadrejeno mapo
`python naloge.py`	`python3 naloge.py`	zagon programa `naloge.py` v Pythonu
`python --version`	`python3 --version`	izpis verzije Pythona

Pri pisanju imen map in datotek v Terminalu lahko vtipkate prvih nekaj črk, nato pa s tipko Tab samodejno dopolnite ime. Če Tab ne deluje, datoteka s tem imenom ni v trenutni mapi.

[4] V macOS so bližnjice odvisne od izbrane tipkovnice, zato jih preverite v aplikaciji *Keyboard Viewer*.

[5] V OS Windows *Command Prompt* ali *PowerShell*, v sistemu macOS pa aplikacija *Terminal*.